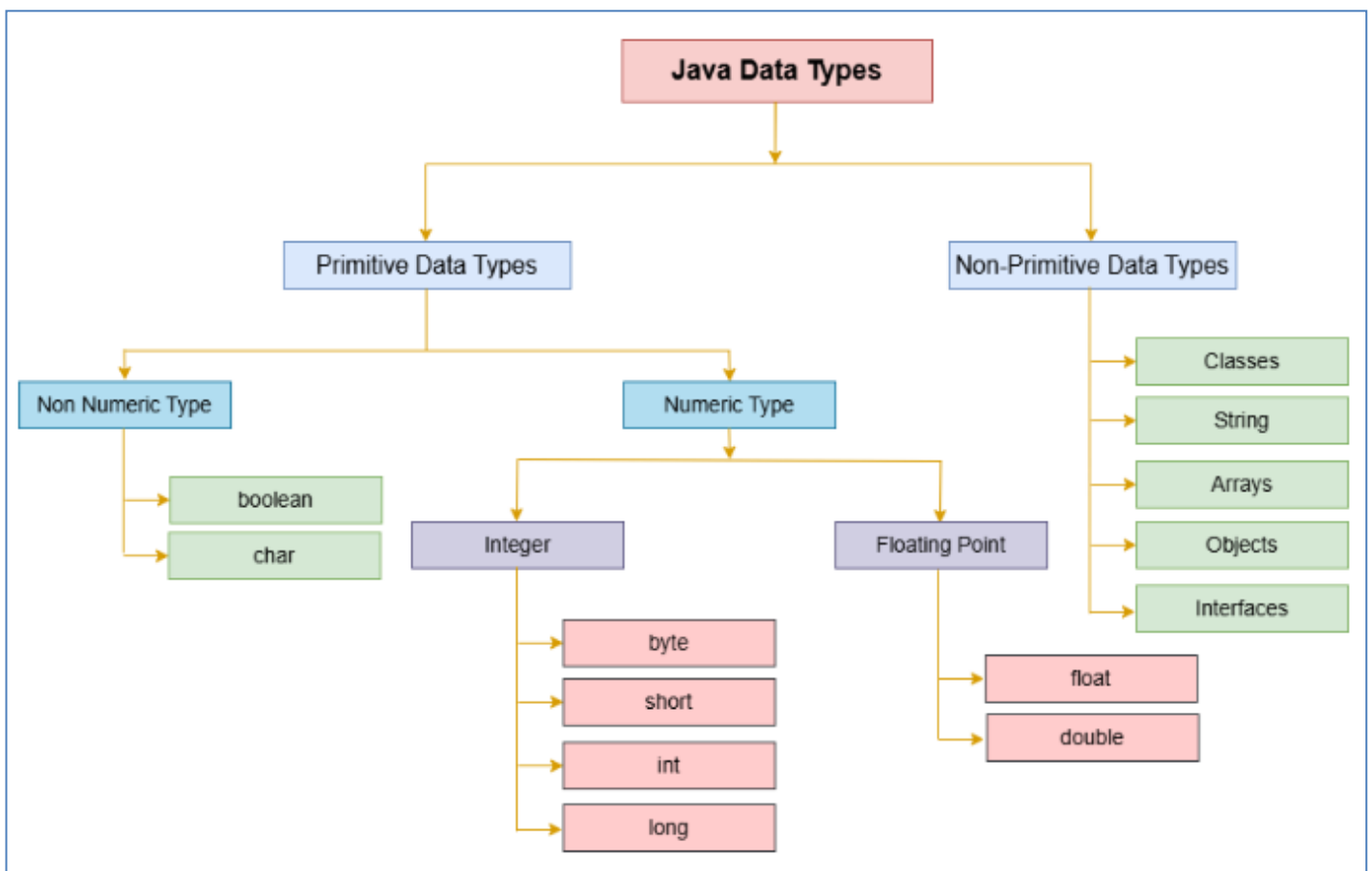


Unit II OOPS Using JAVA

Primitive Data Types:

In programming languages, data types specify the different sizes and values that can be stored in the variable or constants. Each data type is predefined, which makes Java a statically and strongly typed language. There are the following two types of data types in Java.

1. **Primitive Data Types:** The primitive data types include boolean, char, byte, short, int, long, float and double.
2. **Non-Primitive Data Types:** The non-primitive data types include Classes, Interfaces, String, and Arrays.



Data Type	Default Value	Default size
Boolean	False	1 bit
Char	'\u0000'	2 byte
Byte	0	1 byte
Short	0	2 byte

Int	0	4 byte
Long	0L	8 byte
Float	0.0f	4 byte
Double	0.0d	8 byte

or

Java Primitive Data Types

In Java, primitive data types are the basic building blocks for storing data. Java is **statically typed**, so all variables must be declared with a type before use.

The 8 Primitive Types

Type	Size	Default	Range / Notes	Example
boolean	1 byte	false	true / false	boolean flag = true;
byte	8 bits	0	-128 to 127	byte b = 10;
short	16 bits	0	-32,768 to 32,767	short s = 1000;
int	32 bits	0	-2,147,483,648 to 2,147,483,647	int i = 50000;
long	64 bits	0L	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	long l = 5000000L;
float	32 bits	0.0f	~6-7 decimal digits	float f = 3.14f;
double	64 bits	0.0	~15-16 decimal digits	double d = 12.345;
char	16 bits	'\u0000'	Single Unicode character	char c = 'A';

Note:

- **boolean**: stores true/false values.
- **byte / short / int / long**: store integers of varying sizes.
- **float / double**: store decimal numbers; double is more precise.
- **char**: stores a single character using Unicode.

Integers : Java has **4 integer primitive types** for whole numbers:

Type	Size	Range	Example
byte	8 bits	-128 to 127	byte b = 10;
short	16 bits	-32,768 to 32,767	short s = 1000;
int	32 bits	-2,147,483,648 to 2,147,483,647	int i = 50000;
long	64 bits	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	long l = 5000000L;

Key Points:

- Java is **statically typed**; declare type before use.
- Use **byte/short** for small numbers, **int** for normal integers, **long** for very large numbers.
- Supports arithmetic operations: + - * / %.

Example:

```
int sum = 10 + 5000;  
long l = 5000000L;  
System.out.println(sum);  
System.out.println(l);
```

Output:

```
5010  
5000000
```

Floating Point type

Used to store **decimal numbers**. Java has **2 floating-point primitive types**: float and double.

Type	Size	Precision	Example
float	32 bits	~6-7 decimal digits	float f = 3.14f;
double	64 bits	~15-16 decimal digits	double d = 12.34567;

Key Points:

- float requires suffix f or F; double can be used directly.
- double is **default for decimal numbers** in Java.
- Not suitable for exact calculations (e.g., currency); use BigDecimal for precision.

Example:

```
float pi = 3.14f;  
double e = 2.71828;  
System.out.println(pi);  
System.out.println(e);
```

Output:

```
3.14  
2.71828
```

Characters

Character (`char`) in Java

- **Purpose:** Stores a **single character**.
- **Size:** 16 bits (can store Unicode characters).
- **Default Value:** `\u0000`
- **Example Values:** `'A'`, `'z'`, `'0'`, `'%'`

Syntax:

```
char letter = 'A';  
char symbol = '#';
```

Note :

- It Uses **Unicode**, so supports international characters.
- Can perform arithmetic operations (e.g., `'A' + 1` results in 66).

Example:

```
char c = 'A';  
System.out.println(c);    // Output: A  
System.out.println((int)c); // Output: 65 (Unicode value)
```

Booleans

Boolean (`boolean`) in Java

- **Purpose:** Stores **true** or **false** values.
- **Size:** 1 byte (implementation may vary).
- **Default Value:** `false`

Syntax:

```
boolean isJavaFun = true;  
boolean isAdult = false;
```

Note :

- Used for **logical conditions** and **control flow** (if, while).
- Cannot be converted to numeric types directly.

Example:

```
boolean flag = (5 > 3);  
System.out.println(flag); // Output: true
```

User Defined Data Type

User-Defined Data Types in Java

- **Purpose:** Allow programmers to create their **own types** based on classes, interfaces, or enums.
- Unlike primitive types, **user-defined types can have multiple attributes and methods.**
- Useful for modeling real-world entities in programs.

Common User-Defined Types:

- **Class:** Defines objects with attributes (fields) and behavior (methods).

```
class Student {  
    String name;  
    int age;  
}  
  
Student s1 = new Student();  
s1.name = "Alice";  
s1.age = 20;
```

- **Enum:** Represents a fixed set of constants.

```
enum Day { MONDAY, TUESDAY, WEDNESDAY }  
Day today = Day.MONDAY;
```

- **Interface:** Defines a contract that classes can implement.

```
interface Animal {  
    void sound();  
}
```

Note:

- User-defined types allow **code reusability** and **better organization**.
- They are **reference types**, unlike primitives which are value types.

Java Variables

A variable is a container which holds the value while the Java program is executed. A variable is assigned with a data type.

Variable is a name of memory location. There are three types of variables in java: local, instance and static.

Variable

A variable is the name of a reserved area allocated in memory. In other words, it is a name of the memory location. It is a combination of "vary + able" which means its value can be changed.

```
int data=50;//Here data is variable
```

1) Local Variable

A variable declared inside the body of the method is called local variable. You can use this variable only within that method and the other methods in the class aren't even aware that the variable exists.

A local variable cannot be defined with "static" keyword.

2) Instance Variable

A variable declared inside the class but outside the body of the method, is called an instance variable. It is not declared as static.

It is called an instance variable because its value is instance-specific and is not shared among instances.

3) Static variable

A variable that is declared as static is called a static variable. It cannot be local. You can create a single copy of the static variable and share it among all the instances of the class. Memory allocation for static variables happens only once when the class is loaded in the memory.

```
public class A
{
    int m=100;//static variable OR Class Level
    void method()
    {
        int n=90;//local variable
    }
    public static void main(String args[])
    {
        int data=50;//instance variable
    }
}
//end of class
```

Identifiers & Literals

1. Identifiers in Java

- **Definition:** Names given to variables, classes, methods, and other program elements.
- **Rules:**
 1. Can contain letters (A-Z, a-z), digits (0-9), underscore `_`, and dollar `$`.
 2. Must **start with a letter, `_`, or `$`** (cannot start with a digit).
 3. Cannot be a **Java keyword**.
 4. Case-sensitive (`myVar` $\neq$ `MyVar`).
- **Examples:**
 - `int age;`
 - `String studentName;`
 - `final double PI = 3.14;`

2. Literals in Java

- **Definition:** Fixed values assigned to variables in the code.
- **Types of Literals:**
 1. **Integer literals:** `10`, `-25`, `0xFF`
 2. **Floating-point literals:** `3.14`, `0.5f`, `2.5e3`
 3. **Character literals:** `'A'`, `'9'`, `'\n'`
 4. **String literals:** `"Hello"`, `"Java"`
 5. **Boolean literals:** `true`, `false`
 6. **Null literal:** `null` (for reference types)
- **Example:**
 - `int x = 100;       // integer literal`
 - `double pi = 3.14;   // floating-point literal`
 - `char c = 'A';       // character literal`
 - `String name = "Alice";   // string literal`
 - `boolean flag = true;   // boolean literal`

Declarations of constants & variables

In Java, **variables** are used to store data values that can change during program execution, whereas **constants** store fixed values that cannot be changed once assigned.

Variables in Java

A **variable** is a named memory location that holds a value.
It acts as a container for data used in a program.

Syntax:

```
dataType variableName;               // Declaration
dataType variableName = value;       // Declaration + Initialization
```

Example :

```
int age;               // declaration
age = 25;              // initialization
```

```
int marks = 90; // declaration + initialization
```

Rules for Naming Variables

- Variable names are **case-sensitive** (Score $\neq$ score).
- Must **begin with a letter, \$, or _**.
- Cannot begin with a **digit**.
- Cannot use **Java keywords** (like int, class, etc.).
- Should be **meaningful** and follow **camelCase** convention.

Types of Variables in Java

Java variables are classified based on their **scope** and **lifetime**:

Type	Declared Inside	Stored In	Lifetime
Local Variables	Methods, constructors, blocks	Stack memory	Until method ends
Instance Variables	Inside class, outside methods	Heap memory	Till object exists
Static Variables	Inside class with static keyword	Method area	Till program ends

Examples

```
class Student {  
    static String college = "ABC College"; // Static variable  
    String name; // Instance variable  
  
    void display() {  
        int marks = 90; // Local variable  
        System.out.println(marks);  
    }  
}
```

Variable Initialization

Variables **must be initialized** before use.

```
int num = 10; // initialized
```

If a variable is not initialized:

- **Local variables** → must be explicitly initialized.
- **Instance/static variables** → automatically assigned default values.

Data Type	Default Value
byte, short, int, long	0
float, double	0.0
char	'\u0000'
boolean	false
Object references	null

Constants in Java

Definition: A **constant** is a variable whose value **cannot be changed** once assigned.

Keyword: final

Syntax:

final dataType CONSTANT_NAME = value;

Example

```
final double PI = 3.14159;  
final int MAX_STUDENTS = 100;
```

Rules for Declaring Constants

1. Must be declared using the **final** keyword.
2. Should be **initialized at the time of declaration** (or in a constructor if it's an instance constant).
3. Naming convention — constants are usually written in **uppercase letters** with underscores.

Example:

```
final int DAYS_IN_WEEK = 7;  
final double INTEREST_RATE = 6.5;
```

Static Final Constants: When a constant value is **common for all objects**, it is declared as **static final**.

Example

```
class MathConstants {  
  
    static final double PI = 3.14159;  
  
    static final double E = 2.71828;  
  
}
```

These constants can be accessed without creating an object:

```
System.out.println(MathConstants.PI);
```

Difference Between Variables and Constants

Feature	Variable	Constant
Keyword Used	None	final

Value Change	Can be changed	Cannot be changed
Naming Convention	camelCase	UPPERCASE_WITH_UNDERSCORES
Initialization	Optional	Mandatory
Example	int marks = 85;	final int MAX = 100;

Example Program

```
public class ConstantVariableDemo {

    static final double PI = 3.14159; // constant


    public static void main(String[] args) {

        int radius = 5; // variable

        double area = PI * radius * radius;


        System.out.println("Radius: " + radius);

        System.out.println("Area of Circle: " + area);

    }

}
```

Output:

```
Radius: 5
Area of Circle: 78.53975
```

Type Conversion and Casting

Java Type Casting

Type casting is a technique that is used either by the compiler or a programmer to convert one data type to another in Java. Type casting is also known as type conversion. For example, converting int to double, double to int, short to int, etc.

There are two types of type casting allowed in Java programming:

Widening type casting

Narrowing type casting

Widening Type Casting

Widening type casting is also known as implicit type casting in which a smaller type is converted into a larger type, it is done by the compiler automatically.

Hierarchy

Here is the hierarchy of widening type casting in Java:

byte>short>char>int>long>float>double

The compiler plays a role in the type conversion instead of programmers. It changes the type of the variables at the compile time. Also, type conversion occurs from the small data type to large data type only.

Example

In this example, we are adding an integer and a double number, storing the result in a double variable to display the sum, implicitly casting the integer to a double during the addition:

Example :

```
public class Tester {  
    // Main driver method  
  
    public static void main(String[] args) {  
        // Define int variables  
  
        int num1 = 5004;  
  
        double num2 = 2.5;  
  
        double sum = num1 + num2;  
  
        // show output  
  
        System.out.println("The sum of " + num1 + " and " + num2 + " is " + sum);  
    }  
}
```

output : The sum of 5004 and 2.5 is 5006.5

Type Conversion Error

The type conversion error occurs when you try to assign a value of a larger data type to a variable of a smaller data type without explicitly converting it.

Java compiler performs implicit type conversion when assigning values between compatible types, but it does not allow the conversion from a larger data type to a smaller one without explicit type casting.

Example

In the example below, we demonstrated that we can get an error when the compiler tries to convert a large data type to a small data type. Here, we created the num1 integer and num2 double variable. The sum of num1 and num2 will be double, and when we try to store it to the sum of the int type, the compiler gives an error.

```
package com.testing;

public class Tester {

    // Main driver method

    public static void main(String[] args) {

        // Define int variables

        int num1 = 5004;

        double num2 = 2.5;

        int sum = num1 + num2;

        // show output

        System.out.println("The sum of " + num1 + " and " + num2 + " is " + sum);

    }

}
```

Output : Exception in thread "main" java.lang.Error: Unresolved compilation problem:
Type mismatch: cannot convert from double to int

at com.testing.Tester.main(Tester.java:9)

Narrowing Type Casting

Narrowing type casting is also known as explicit type casting or explicit type conversion which is done by the programmer manually. In the narrowing type casting a larger type can be converted into a smaller type.

When a programmer changes the variable type while writing the code. We can use the cast operator to change the type of the variable. For example, double to int or int to double.

Syntax

Below is the syntax for narrowing type casting i.e., to manually type conversion:

```
double doubleNum = (double) num;
```

The above code statement will convert the variable to double type.

Example

In the example below, we define the num variable of integer type and initialize it with the value. Also, we define the doubleNum variable of double type and store the num variable's value after converting it to the double.

Next, We created the 'convertedInt' integer type variable and stored the double value after type casting to int. In the output, we can observe the value of the double and int variables.

```
public class Tester {  
    // Main driver method  
    public static void main(String[] args) {  
        // Define int variable  
        int num = 5004;  
        // Type casting int to double  
        double doubleNum = (double) num;  
        // show output  
        System.out.println("The value of " + num + " after converting to the double is " +  
doubleNum);  
        // Type casting double to int  
        int convertedInt = (int) doubleNum;  
        // show output  
        System.out.println("The value of " + doubleNum + " after converting to the int again is " +  
convertedInt);  
    }  
}
```

Output :

The value of 5004 after converting to the double is 5004.0

The value of 5004.0 after converting to the int again is 5004

Difference between Type Casting and Type Conversion

1. **Type Casting:** In type casting, a data type is converted into another data type by the programmer using the casting operator during the program design. In type casting, the destination data type may be smaller than the source data type when converting the data type to another data type, that's why it is also called narrowing conversion.

Syntax/Declaration:-

```
destination_datatype = (target_datatype)variable;
```

() : is a casting operator.

target_datatype: is a data type in which we want to convert the source data type.

Type Casting example -

```
float x;
```

```
byte y;
```

```
...
```

```
...
```

```
y=(byte)x; //Line 5
```

In Line 5: you can see that, we are converting float(source) data type into byte(target) data type.

2. Type conversion : In type conversion, a data type is automatically converted into another data type by a compiler at the compiler time. In type conversion, the destination data type cannot be smaller than the source data type, that's why it is also called widening conversion. One more important thing is that it can only be applied to compatible data types.

Type Conversion example -

```
int x=30;  
float y;  
y=x; // y==30.000000.
```

S.NO	TYPE CASTING	TYPE CONVERSION
1.	In type casting, a data type is converted into another data type by a programmer using casting operator.	Whereas in type conversion, a data type is converted into another data type by a compiler.
2.	Type casting can be applied	Whereas type conversion can only be

	to compatible data types as well as incompatible data types .	applied to compatible datatypes .
3.	In type casting, casting operator is needed in order to cast a data type to another data type.	Whereas in type conversion, there is no need for a casting operator.
4.	In type casting, the destination data type may be smaller than the source data type, when converting the data type to another data type.	Whereas in type conversion, the destination data type can't be smaller than source data type.
5.	Type casting takes place during the program design by programmer.	Whereas type conversion is done at the compile time.
6.	Type casting is also called narrowing conversion because in this, the destination data type may be smaller than the source data type.	Whereas type conversion is also called widening conversion because in this, the destination data type can not be smaller than the source data type.
7.	Type casting is often used in coding and competitive programming works.	Whereas type conversion is less used in coding and competitive programming as it might cause incorrect answer.
8.	Type casting is more efficient and reliable.	Whereas type conversion is less efficient and less reliable.

Scope of variables & default values of variables declared

In Java, **variables** are declared inside classes, methods, or blocks, and each variable has a **scope** — the region of the program where it can be **accessed or used**.

Also, variables may have **default values** depending on where and how they are declared.

Note:

- Local variables **must be initialized manually**.
- Instance and static variables get **default values automatically**.
- **Scope** determines where a variable can be used.
- **Lifetime** defines how long a variable exists in memory.
- Static variables are **shared**, while instance variables are **unique** to each object.

Variable Type	Scope	Default Value	Keyword Used	Example Declaration
Local Variable	Inside methods/blocks	No default value	—	int x = 10;
Instance Variable	Inside class, outside methods	Assigned by JVM	—	int age;
Static Variable	Inside class, declared with static	Assigned by JVM	static	static int count;

Scope of Variables:

The **scope of a variable** refers to the **area or lifetime** within a program where the variable is **visible and accessible**.

Types of Variable Scopes in Java

Type of Variable	Where Declared	Scope (Accessible Area)	Lifetime
Local Variable	Inside methods, constructors, or blocks	Only within that method or block	Until method/block execution ends
Instance Variable	Inside a class but outside any method	Accessible by all methods of that object	As long as the object exists
Static Variable (Class Variable)	Inside a class, declared using <code>static</code>	Shared by all objects of the class	As long as the program runs

Local Variables

- Declared **inside** a method, constructor, or block.
- **Created** when the method is invoked and **destroyed** when it ends.
- Must be **initialized manually** before use.
- **No default value** is assigned.

Example:

```
public class Example {  
  
    void display() {  
  
        int num = 10;    // local variable  
  
        System.out.println(num);  
  
    }  
  
}
```

Note:

If you try to use `num` outside `display()` method, it will cause an **error**.

Instance Variables

- Declared **inside a class**, but **outside any method or block**.
- Each object of the class gets its **own copy** of these variables.
- Automatically assigned **default values** if not initialized.
- Accessed using **object reference**.

Example:

```
public class Student {
```

```
String name; // instance variable
```

```
int age; // instance variable
```

```
void show() {
```

```
    System.out.println("Name: " + name + ", Age: " + age);
```

```
}
```

```
}
```

Output : Name: null, Age: 0

Static Variables (Class Variables)

- Declared with the **static** keyword.
- **Shared among all objects** of the class.
- Stored in the **method area (class memory)**.
- Initialized **only once** when the class is loaded.
- Have **default values**.

Example:

```
public class Student {
```

```
    static String college = "ABC College"; // static variable
```

```
    String name;
```

```
    void show() {
```

```
        System.out.println(name + " studies in " + college);
```

```
    }
```

```
}
```

Block Scope (Within Loops / Conditional Blocks)

Variables declared inside a block {} exist only **within that block**.

Example:

```
public class Test {
```

```
    public static void main(String[] args) {
```

```
        if (true) {
```

```

        int x = 5;

        System.out.println(x); // valid
    }

    // System.out.println(x); // Error: x cannot be resolved
}
}

```

Variable Lifetime

Type	When Created	When Destroyed
Local	When method/block starts	When method/block ends
Instance	When object is created	When object is destroyed
Static	When class is loaded	When program ends

Default Values of Variables

Local Variables

- **No default value** is assigned automatically.
- Must be explicitly **initialized before use**.

Example:

```

public class Test {

    public static void main(String[] args) {

        int x;

        // System.out.println(x); // Error: variable x might not have been initialized
    }

}

```

How to pass command line argument

Command-line arguments are values passed to a Java program **at the time of execution**. They allow users to **input data from the command prompt** (without using Scanner).

2. Syntax

Arguments are passed **after the class name** during execution:

```
java ClassName arg1 arg2 arg3 ...
```

In the program, they are received through the `String args[]` parameter in the `main()` method.

3. Example

```
public class CommandExample {  
    public static void main(String[] args) {  
        System.out.println("First Argument: " + args[0]);  
        System.out.println("Second Argument: " + args[1]);  
    }  
}
```

4. How to Run

1. **Compile the program:**
2. `javac CommandExample.java`
3. **Run the program with arguments:**
4. `java CommandExample Hello Java`

Output:

```
First Argument: Hello  
Second Argument: Java
```

5. Important Points

- `args[]` is an **array of Strings**.
- The **first argument** is stored in `args[0]`.
- Arguments are **always treated as strings** — convert them if needed:
- `int num = Integer.parseInt(args[0]);`
- If no arguments are passed, accessing `args[0]` causes an **`ArrayIndexOutOfBoundsException`**.

Example:

```
public class Sum {  
    public static void main(String[] args) {  
        int a = Integer.parseInt(args[0]);  
        int b = Integer.parseInt(args[1]);  
        System.out.println("Sum = " + (a + b));  
    }  
}
```

Run:

```
java Sum 10 20
```

Output:

Sum = 30

How to take input from user in java using scanner class

The **Scanner class** in Java is used to **take input** from the **user** (keyboard). It is present in the **java.util** package.

2. Syntax

```
import java.util.Scanner;
```

```
Scanner sc = new Scanner(System.in);
```

- `System.in` → standard input stream (keyboard)
- `sc` → object of Scanner class

3. Common Input Methods

Method	Description	Example
<code>nextInt()</code>	Reads an integer	<code>int a = sc.nextInt();</code>
<code>nextFloat()</code>	Reads a float value	<code>float b = sc.nextFloat();</code>
<code>next()</code>	Reads a single word (String)	<code>String name = sc.next();</code>
<code>nextLine()</code>	Reads a complete line	<code>String line = sc.nextLine();</code>
<code>nextDouble()</code>	Reads a double value	<code>double d = sc.nextDouble();</code>
<code>nextBoolean()</code>	Reads a boolean value	<code>boolean flag = sc.nextBoolean();</code>

4. Example

```
import java.util.Scanner;
```

```
public class UserInputExample {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in); // Create scanner object  
  
        System.out.print("Enter your name: ");  
        String name = sc.nextLine(); // take string input  
  
        System.out.print("Enter your age: ");  
        int age = sc.nextInt(); // take integer input  
  
        System.out.println("Hello " + name + ", you are " + age + " years old.");  
  
        sc.close(); // close the scanner  
    }  
}
```

Output

Enter your name: ABC
Enter your age: 21
Hello ABC, you are 21 years old.

5. Important Points

- Always **import** java.util.Scanner;
- **Close** the scanner using sc.close() after use.
- Avoid mixing nextLine() with other input methods without handling the newline (\n).

Wrapper classes

The **wrapper class in Java** provides the mechanism to *convert primitive into object and object into primitive*.

Since J2SE 5.0, **autoboxing** and **unboxing** feature convert primitives into objects and objects into primitives automatically. The automatic conversion of primitive into an object is known as autoboxing and vice-versa unboxing.

Advantages of Wrapper Classes

Java Wrapper classes offer several advantages over primitive data types. It includes encapsulation, type conversion, autoboxing and unboxing, and utility methods. Wrapper classes are also thread-safe and immutable, making them ideal for multi-threaded applications. Some advantages of wrapper class are:

1. Collections allowed only object data.
2. On object data we can call multiple methods compareTo(), equals(), toString()
3. Cloning process only objects
4. Object data allowed null values.
5. Serialization can allow only object data.

Significance of Wrapper Class in Java

- Encapsulation
- Type Conversion
- Utility Methods
- Null Handling

Use of Wrapper Classes in Java

Java is an object-oriented programming language, so we need to deal with objects many times like in Collections, Serialization, Synchronization, etc. Let us see the different scenarios, where we need to use the wrapper classes.

- **Change the value in Method:** Java supports only call by value. So, if we pass a primitive value, it will not change the original value. But, if we convert the primitive value in an object, it will change the original value.

- **Serialization:** We need to convert the objects into streams to perform the serialization. If we have a primitive value, we can convert it in objects through the wrapper classes.
- **Synchronization:** Java synchronization works with objects in Multithreading.
- **java.util package:** The java.util package provides the utility classes to deal with objects.
- **Collection Framework:** Java collection framework works with objects only. All classes of the collection framework (ArrayList, LinkedList, Vector, HashSet, LinkedHashSet, TreeSet, PriorityQueue, ArrayDeque, etc.) deal with objects only.

The eight classes of the java.lang package are known as wrapper classes in Java. The list of eight wrapper classes is given below:

Primitive Type	Wrapper class
boolean	<u>Boolean</u>
char	<u>Character</u>
byte	<u>Byte</u>
short	<u>Short</u>
int	<u>Integer</u>
long	<u>Long</u>
float	<u>Float</u>
double	<u>Double</u>

Autoboxing

The automatic conversion of primitive data type into its corresponding wrapper class is known as autoboxing, for example, byte to Byte, char to Character, int to Integer, long to Long, float to Float, boolean to Boolean, double to Double, and short to Short.

Since Java 5, we do not need to use the `valueOf()` method of wrapper classes to convert the primitive into objects.

Wrapper Class Example: Primitive to Wrapper

Example

1. *//Java program to convert primitive into objects*
2. *//Autoboxing example of int to Integer*
3. **public class** WrapperExample1{
4. **public static void** main(String args[]){
5. *//Converting int into Integer*
6. **int** a=20;
7. **Integer** i=Integer.valueOf(a);*//converting int into Integer explicitly*
8. **Integer** j=a;*//autoboxing, now compiler will write Integer.valueOf(a) internally*
- 9.

```
10. System.out.println(a+" "+i+" "+j);
11. }}
```

Output:

20 20 20

Unboxing

The automatic conversion of wrapper type into its corresponding primitive type is known as unboxing. It is the reverse process of autoboxing. Since Java 5, we do not need to use the `intValue()` method of wrapper classes to convert the wrapper type into primitives.

Wrapper Class Example: Wrapper to Primitive

File Name: WrapperExample2.java

```
1. //Java program to convert object into primitives
2. //Unboxing example of Integer to int
3. public class WrapperExample2{
4.     public static void main(String args[]){
5.         //Converting Integer to int
6.         Integer a=new Integer(3);
7.         int i=a.intValue();//converting Integer to int explicitly
8.         int j=a;//unboxing, now compiler will write a.intValue() internally
9.
10.        System.out.println(a+" "+i+" "+j);
11.    }}
```

Output:

3 3 3

Java Wrapper Classes Example

File Name: WrapperExample3.java

```
1. //Java Program to convert all primitives into its corresponding
2. //wrapper objects and vice-versa
3. public class WrapperExample3{
4.     public static void main(String args[]){
5.         byte b=10;
6.         short s=20;
7.         int i=30;
8.         long l=40;
9.         float f=50.0F;
10.        double d=60.0D;
11.        char c='a';
12.        boolean b2=true;
13.
14.        //Autoboxing: Converting primitives into objects
15.        Byte byteobj=b;
16.        Short shortobj=s;
17.        Integer intobj=i;
18.        Long longobj=l;
```

```

19. Float floatobj=f;
20. Double doubleobj=d;
21. Character charobj=c;
22. Boolean boolobj=b2;
23.
24. //Printing objects
25. System.out.println("---Printing object values---");
26. System.out.println("Byte object: "+byteobj);
27. System.out.println("Short object: "+shortobj);
28. System.out.println("Integer object: "+intobj);
29. System.out.println("Long object: "+longobj);
30. System.out.println("Float object: "+floatobj);
31. System.out.println("Double object: "+doubleobj);
32. System.out.println("Character object: "+charobj);
33. System.out.println("Boolean object: "+boolobj);
34.
35. //Unboxing: Converting Objects to Primitives
36. byte bytevalue=byteobj;
37. short shortvalue=shortobj;
38. int intvalue=intobj;
39. long longvalue=longobj;
40. float floatvalue=floatobj;
41. double doublevalue=doubleobj;
42. char charvalue=charobj;
43. boolean boolvalue=boolobj;
44.
45. //Printing primitives
46. System.out.println("---Printing primitive values---");
47. System.out.println("byte value: "+bytevalue);
48. System.out.println("short value: "+shortvalue);
49. System.out.println("int value: "+intvalue);
50. System.out.println("long value: "+longvalue);
51. System.out.println("float value: "+floatvalue);
52. System.out.println("double value: "+doublevalue);
53. System.out.println("char value: "+charvalue);
54. System.out.println("boolean value: "+boolvalue);
55. }}

```

Output:

```

---Printing object values---
Byte object: 10
Short object: 20
Integer object: 30
Long object: 40
Float object: 50.0
Double object: 60.0
Character object: a
Boolean object: true
---Printing primitive values---
byte value: 10
short value: 20
int value: 30
long value: 40
float value: 50.0
double value: 60.0
char value: a
boolean value: true

```

Custom Wrapper Class in Java

Java Wrapper classes wrap the primitive data types, that is why it is known as wrapper classes. We can also create a class which wraps a primitive data type. So, we can create a custom wrapper class in Java.

File Name: Test.java

```
1. //Creating the custom wrapper class
2. class Test{
3.     private int i;
4.     Test(){ }
5.     Test(int i){
6.         this.i=i;
7.     }
8.     public int getValue(){
9.         return i;
10.    }
11.    public void setValue(int i){
12.        this.i=i;
13.    }
14.    @Override
15.    public String toString() {
16.        return Integer.toString(i);
17.    }
18. }
19. //Testing the custom wrapper class
20. public class TestTest{
21.     public static void main(String[] args){
22.         Test j=new Test(10);
23.         System.out.println(j);
24.     }}
```

Output:

10

Considerations Java Wrapper Classes Vs. Primitive Types

Discuss considerations like memory usage, performance, and when to choose one over the other.

Wrapper Classes	Primitive Types	Considerations
Memory Usage	Low	High due to lack of overhead
Performance	Moderate	High because of fewer instructions
When to Use	When Encapsulation is needed or when working with complex APIs	When Performance and Memory are a major concern
Security	High	Low as primitives are not immutable

Thread Safety	Yes	No
---------------	-----	----

Comment Syntax

Comments are lines in code that are **not executed** by the compiler. They are used to **explain code** and **improve readability**.

2. Types of Comments in Java

Type	Syntax	Description
Single-line Comment	// comment text	Used for short comments or notes.
Multi-line Comment	/* comment text */	Used for longer explanations or to comment out multiple lines.
Documentation Comment	/** comment text */	Used to create documentation using the Javadoc tool .

3. Examples

Single-line Comment

```
// This is a single-line comment
int x = 10; // assigning value to x
```

Multi-line Comment

```
/* This is a multi-line comment.
   It can span across multiple lines. */
int y = 20;
```

Documentation Comment

```
/**
 * This class performs addition of two numbers.
 * @author ABC
 */
public class Addition {
    public static void main(String[] args) {
        int a = 5, b = 10;
        System.out.println(a + b);
    }
}
```

Note:

- Comments **do not affect program execution**.
- Used for **code explanation**, **debugging**, and **documentation**.
- **/** ... */** is mainly used for **API documentation**.

Garbage Collection

Java Garbage Collection

In java, garbage means unreferenced objects.

In **Java**, **garbage collection** is the process of managing memory, automatically. It finds the unused objects (that are no longer used by the program) and delete or remove them to free up the memory. The **garbage collection** mechanism uses several GC algorithms.

Garbage Collection is process of reclaiming the runtime unused memory automatically. In other words, it is a way to destroy the unused objects.

To do so, we were using free() function in C language and delete() in C++. But, in java it is performed automatically. So, java provides better memory management.

What does Java Garbage Collector?

JVM controls the garbage collector. JVM decides when to perform the garbage collection. You can also request to the JVM to run the garbage collector. But there is no guarantee under any conditions that the JVM will comply. JVM runs the garbage collector if it senses that memory is running low. When Java program request for the garbage collector, the JVM usually grants the request in short order. It does not make sure that the requests accept.

Note: The GC of JVM collects only those objects that are created by new keywords. So if you have created any objects without new, you can use finalize method to perform cleanup processing (destroying remaining objects).

Advantage of Garbage Collection

- It makes java **memory efficient** because garbage collector removes the unreferenced objects from heap memory.
- It is **automatically done** by the garbage collector (a part of JVM) so we don't need to make extra efforts.

finalize() method

The finalize() method is invoked each time before the object is garbage collected. This method can be used to perform cleanup processing. This method is defined in Object class as:

```
protected void finalize(){}
```

gc() method

The gc() method is used to invoke the garbage collector to perform cleanup processing. The gc() is found in System and Runtime classes.

```
public class TestGarbage1{  
    public void finalize()  
    {  
        System.out.println("object is garbage collected");  
    }  
    public static void main(String args[]){  
        TestGarbage1 s1=new TestGarbage1();  
        TestGarbage1 s2=new TestGarbage1();  
        s1=null;  
        s2=null;  
        System.gc();  
    }  
}
```

OUTPUT:

object is garbage collected
object is garbage collected

[Arrays of Primitive Data Types: Types of Arrays, Creation, concatenation and conversion of a string,]

i. Array :

An array is typically a grouping of elements of the same kind that are stored in a single, contiguous block of memory.

Java array is an object which contains elements of a similar data type. Additionally, The elements of an array are stored in a contiguous memory location. It is a data structure where we store similar elements. We can store only a fixed set of elements in a Java array.

Array in Java is index-based, the first element of the array is stored at the 0th index, 2nd element is stored on 1st index and so on.

In contrast to C/C++, the length member allows us to obtain the array's length. We must utilise the sizeof operator in C/C++.

An object of a dynamically formed class is called an array in Java. Java arrays implement the Serializable and Cloneable interfaces and derive from the Object class. In Java, an array can hold objects or primitive values. Java allows us to create single- or multi-dimensional arrays, just like C/C++ does.

Additionally, C/C++ does not support the anonymous array functionality that Java does.

Advantages

- **Code Optimization:** It makes the code optimized, we can retrieve or sort the data efficiently.
- **Random access:** We can get any data located at an index position.

Disadvantages

- **Size Limit:** Arrays have a fixed size and do not grow dynamically at runtime.

Passing Array to a Method in Java

We can pass the Java array to the method so that we can reuse the same logic on any array. When we pass an array to a method in Java, we are essentially passing a reference to the array. It means that the method will have access to the same array data as the calling code, and any modifications made to the array within the method will affect the original array.

Let's see the simple example to get the minimum number of an array using a method.

Example

```
1. //Java Program to demonstrate the way of passing an array to method.
2. public class Main{
3.     //creating a method which receives an array as a parameter
4.     static void min(int arr[]){
5.         int min=arr[0];
6.         for(int i=1;i<arr.length;i++)
7.             if(min>arr[i])
8.                 min=arr[i];
9.
10.        System.out.println(min);
11.    }
12.
13.    public static void main(String args[]){
14.        int a[]={33,3,4,5};//declaring and initializing an array
15.        min(a);//passing array to method
16.    }
17. }
```

Output:

3

Explanation

This Java program demonstrates the concept of passing an array to a method. The min method takes an integer array arr as a parameter and finds the minimum element in the array using a simple iterative loop. In the main method, an integer array a is declared and initialized with values {33, 3, 4, 5}, and then the min method is called with this array as an argument. The min method iterates through the array to find the minimum element and prints it to the console.

ArrayIndexOutOfBoundsException

In Java, the ArrayIndexOutOfBoundsException is a runtime exception thrown by the Java Virtual Machine (JVM) when attempting to access an invalid index of an array. This exception occurs if the index is negative, equal to the size of the array, or greater than the size of the array while traversing the array.

TestArrayException.java

```
1. //Java Program to demonstrate the case of
2. //ArrayIndexOutOfBoundsException in a Java Array.
3. public class TestArrayException{
4.     public static void main(String args[]){
5.         int arr[]={50,60,70,80};
6.         for(int i=0;i<=arr.length;i++){
7.             System.out.println(arr[i]);
8.         }
9.     }}
```

Output:

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 4
at TestArrayException.main(TestArrayException.java:5)
50
60
70
80
```

ii. Array creation :

Declaration, Instantiation and Initialization of Java Array

In Java, you can declare, instantiate, and initialize an array in a single line, as demonstrated below:

1. `int a[]={33,3,4,5};`//declaration, instantiation and initialization
Let's see the simple example to print this array.

Example

1. *//Java Program to illustrate the use of declaration, instantiation*
2. *//and initialization of Java array in a single line*
3. `public class` Main{
4. `public static void` main(String args[]){
5. *//declaration, instantiation and initialization of an array*
6. `int` a[]={33,3,4,5};
7. *//traversing array*
8. `for(int` i=0;i<a.length;i++)//length is the property of array
9. `System.out.println(a[i]);`
10. }
11. }

Output:

```
33
3
4
5
```

iii. Types of Array :

Types of Array in java

There are two types of array.

- Single Dimensional Array
 - Multidimensional Array
-

Single-Dimensional Array in Java

A single-dimensional array in Java is a linear collection of elements of the same data type. It is declared and instantiated using the following syntax:

1. dataType[] arr; (or)
2. dataType []arr; (or)
3. dataType arr[];

Instantiation of an Array in Java

1. arrayRefVar=**new** datatype[size];

Example of Java Array

Let's see the simple example of java array, where we are going to declare, instantiate, initialize and traverse an array.

Example

1. *//Java Program to illustrate how to declare, instantiate, initialize*
2. *//and traverse the Java array.*
3. **public class** Main{
4. **public static void** main(String args[]){
5. *//declaration and instantiation of an array*
6. **int** a[]=**new int**[5];
7. a[0]=10;*//initialization*
8. a[1]=20;
9. a[2]=70;
10. a[3]=40;
11. a[4]=50;
12. *//traversing array*
13. **for**(**int** i=0;i<a.length;i++){*//length is the property of array*
14. System.out.println(a[i]);
15. }
16. }
17. }

Output:

10

20

70

40

50

Multidimensional Array in Java

A multidimensional array in Java is an array of arrays where each element can be an array itself. It is useful for storing data in row and column format.

Syntax to Declare Multidimensional Array in Java

1. dataType[][] arrayRefVar; (or)
2. dataType [][]arrayRefVar; (or)
3. dataType arrayRefVar[][]; (or)
4. dataType []arrayRefVar[];

Example to instantiate Multidimensional Array in Java

1. `int[][] arr=new int[3][3];`*//3 row and 3 column*

Example to initialize Multidimensional Array in Java

1. `arr[0][0]=1;`
2. `arr[0][1]=2;`
3. `arr[0][2]=3;`
4. `arr[1][0]=4;`
5. `arr[1][1]=5;`
6. `arr[1][2]=6;`
7. `arr[2][0]=7;`
8. `arr[2][1]=8;`
9. `arr[2][2]=9;`

Example of Multidimensional Java Array

Let's see the simple example to declare, instantiate, initialize and print the 2Dimensional array.

Example

1. `public class` Main {
2. `public static void` main(String args[]) {
3. `int` arr[][] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}}; *// 3x3 matrix*
4. *// Printing the 2D array*
5. `for` (`int` i = 0; i < 3; i++) {
6. `for` (`int` j = 0; j < 3; j++) {
7. System.out.print(arr[i][j] + " ");
8. }
9. System.out.println();
10. }
11. }
12. }

Output:

```
1 2 3
4 5 6
7 8 9
```

Explanation

This Java program initializes and prints a 2D array, representing a 3x3 matrix. Initially, a 2D array named arr is declared and initialized with values using array initializer syntax. The

array consists of three rows, each containing three columns. The program then iterates through each row and column of the array using nested loops. Within the loops, it prints the value of each element, separated by a space. After printing all the elements of a row, a newline character is printed to move to the next line. This process continues until all elements of the array are printed. As a result, the program outputs the 3x3 matrix with each element displayed in its respective row and column.

These examples demonstrate the declaration, instantiation, initialization, and traversal of both single-dimensional and multidimensional arrays in Java.

Jagged Arrays in Java

In Java, a jagged array is an array of arrays where each row of the array can have a different number of columns. This contrasts with a regular two-dimensional array, where each row has the same number of columns.

Declaring and Initializing a Jagged Array

To declare a jagged array in Java, you first declare the array of arrays, and then you initialize each row separately with its own array of columns.

1. `int arr[][] = new int[3][];`
2. `arr[0] = new int[3];`
3. `arr[1] = new int[4];`
4. `arr[2] = new int[2];`

Example

1. *//Java Program to illustrate the jagged array*
2. `public class Main{`
3. `public static void main(String[] args){`
4. *//declaring a 2D array with odd columns*
5. `int arr[][] = new int[3][];`
6. `arr[0] = new int[3];`
7. `arr[1] = new int[4];`
8. `arr[2] = new int[2];`
9. *//initializing a jagged array*
10. `int count = 0;`
11. `for (int i=0; i<arr.length; i++){`
12. `for(int j=0; j<arr[i].length; j++){`
13. `arr[i][j] = count++;`
14. `}`
15. *//printing the data of a jagged array*
16. `for (int i=0; i<arr.length; i++){`
17. `for (int j=0; j<arr[i].length; j++){`
18. `System.out.print(arr[i][j]+" ");`
19. `}`
20. `System.out.println();//new line`
21. `}`
22. `}`
23. `}`

Output:

```
0 1 2
3 4 5 6
7 8
```

Explanation

This Java program demonstrates the concept of a jagged array, where an array of arrays is created, with each inner array having a different number of columns. Initially, a 2D array named `arr` is declared with 3 rows, but with each row having a different number of columns: the first row has 3 columns, the second row has 4 columns, and the third row has 2 columns. The program then initializes the jagged array by filling it with sequential values starting from 0. Finally, it iterates through the array and prints out each element, row by row, separated by a space, with a newline character indicating the end of each row.

What is the class name of a Java array?

In Java, an array is an object. However, arrays have a special feature in Java where the JVM generates a proxy class for each array object. This proxy class represents the runtime type of the array. To obtain the class name of a Java array, you can use the `getClass().getName()` method, which is available on all Java objects. This method returns a `String` representing the fully qualified name of the class of the object, including any package information.

Example

```
1. //Java Program to get the class name of array in Java
2. public class Main{
3.     public static void main(String args[]){
4.         //declaration and initialization of array
5.         int arr[]={4,4,5};
6.         //getting the class name of Java array
7.         Class c=arr.getClass();
8.         String name=c.getName();
9.         //printing the class name of Java array
10.        System.out.println(name);
11.    }
12. }
```

Output:

```
[I
```

Explanation

This Java program demonstrates how to retrieve the class name of an array in Java using the `getClass().getName()` method. It initializes an array `arr` with integer values, obtains the runtime class of the array using `getClass()`, and retrieves the class name using `getName()`. The obtained class name, typically represented by a single character followed by square brackets, reflects the dimensionality and type of elements in the array.

iv. Array concatenation:

Array concatenation means **joining two or more arrays** to form a **single array** containing all elements of the original arrays.

2. Example (Using Loops)

```
public class Main {
    public static void main(String[] args) {
        int[] a = {1, 2, 3};
        int[] b = {4, 5, 6};

        int[] c = new int[a.length + b.length];

        // Copy elements of first array
        for(int i = 0; i < a.length; i++) {
            c[i] = a[i];
        }

        // Copy elements of second array
        for(int i = 0; i < b.length; i++) {
            c[a.length + i] = b[i];
        }

        // Display concatenated array
        for(int i : c) {
            System.out.print(i + " ");
        }
    }
}
```

Output:

1 2 3 4 5 6

3. Using `System.arraycopy()`

```
int[] c = new int[a.length + b.length];
System.arraycopy(a, 0, c, 0, a.length);
System.arraycopy(b, 0, c, a.length, b.length);
```

4. Using Java 8 Streams

```
int[] c = java.util.stream.IntStream.concat(
    java.util.Arrays.stream(a),
    java.util.Arrays.stream(b))
    .toArray();
```

Note:

Java arrays are **fixed in size**, so concatenation creates a **new array** instead of modifying existing ones.

OR

Copying a Java Array

Copying an array in Java can be achieved using the `arraycopy()` method of the `System` class. This method allows you to copy elements from a source array to a destination array with a specified starting position and length.

Syntax of arraycopy method

1. **public static void** arraycopy(
2. Object src, **int** srcPos, Object dest, **int** destPos, **int** length
3.)

Here, `src` is the source array, `srcPos` is the starting position in the source array, `dest` is the destination array, `destPos` is the starting position in the destination array, and `length` is the number of elements to be copied.

Example

1. *//Java Program to copy a source array into a destination array in Java*
2. **public class** Main {
3. **public static void** main(String[] args) {
4. *//declaring a source array*
5. **char**[] copyFrom = { 'd', 'e', 'c', 'a', 'f', 'f', 'e',
6. 'i', 'n', 'a', 't', 'e', 'd' };
7. *//declaring a destination array*
8. **char**[] copyTo = **new char**[7];
9. *//copying array using System.arraycopy() method*
10. System.arraycopy(copyFrom, 2, copyTo, 0, 7);
11. *//printing the destination array*
12. System.out.println(String.valueOf(copyTo));
13. }
14. }

Output:

caffein

Explanation

The Java program initializes two character arrays, `copyFrom` and `copyTo`, with predefined values. It then utilizes the `System.arraycopy()` method to copy a portion of the `copyFrom` array into the `copyTo` array, starting from index 2 and copying 7 elements. Finally, it prints the contents of the `copyTo` array, resulting in the output "caffein".

Cloning an Array in Java

In Java, arrays implement the `Cloneable` interface, allowing us to create clones of arrays. If we create a clone of a single-dimensional array, it creates a deep copy of the Java array. It means it will copy the actual value. But, if we create the clone of a multidimensional array, it creates a shallow copy of the Java array, which means it copies the references.

This distinction is important because modifying elements in a shallow copied multidimensional array will affect both the original and cloned arrays, while in a deep copied single-dimensional array, modifications will not impact the original.

TestarrayClone.java

```
1. //Java Program to clone the array
2. class TestarrayClone{
3. public static void main(String args[]){
4. int arr[]={33,3,4,5};
5. System.out.println("Printing original array:");
6. for(int i:arr)
7. System.out.println(i);
8. System.out.println("Printing clone of the array:");
9. int carr[]=arr.clone();
10. for(int i:carr)
11. System.out.println(i);
12. System.out.println("Are both equal?");
13. System.out.println(arr==carr);
14. }
15. }
```

Output:

Printing original array:

33
3
4
5

Printing clone of the array:

33
3
4
5

Are both equal?

False

Explanation

An integer array called arr is initialised with the numbers {33, 3, 4, 5} in this Java programme, and its elements are printed. Then, it uses the clone() method to duplicate arr, assigns the result to carr, and prints the items of the duplicated array. The last line of the programme compares arr and carr using the == operator; the result evaluates to false, meaning that arr and carr are distinct array objects. The elements in both arrays, though, are the same.

v. conversion of string :

String conversion means changing data from one type to a string or converting a string to another data type.

Conversion Type	Method
-----------------	--------

<code>int → String</code>	<code>String.valueOf(int)</code>
<code>String → int</code>	<code>Integer.parseInt(String)</code>
<code>double → String</code>	<code>String.valueOf(double)</code>
<code>String → double</code>	<code>Double.parseDouble(String)</code>
<code>String → char[]</code>	<code>toCharArray()</code>
<code>char[] → String</code>	<code>new String(char[])</code>

Converting Other Types → String

Using `String.valueOf()`

```
int num = 100;
String str = String.valueOf(num);
System.out.println(str); // Output: "100"
```

Using `Integer.toString()`

```
int num = 50;
String str = Integer.toString(num);
```

Using String Concatenation

```
int num = 25;
String str = num + "";
```

3. Converting String → Other Types

String to int

```
String s = "123";
int num = Integer.parseInt(s);
```

String to double

```
String s = "12.5";
double d = Double.parseDouble(s);
```

String to boolean

```
String s = "true";
boolean b = Boolean.parseBoolean(s);
```

4. Converting String to char array

```
String s = "Hello";
char[] ch = s.toCharArray();
```

5. Converting char array to String

```
char[] ch = {'J','a','v','a'};
String s = new String(ch);
```

Arrays of Primitive Data Types in Java

An **array of primitive data types** is used to **store multiple values** of the **same primitive type** (like int, char, float, etc.) in a **single variable**.

Primitive data types include:

byte, short, int, long, float, double, char, and boolean.

2. Syntax

```
datatype[] arrayName = new datatype[size];
```

or

```
datatype arrayName[] = new datatype[size];
```

3. Example

```
int[] numbers = new int[5]; // array of int
numbers[0] = 10;
numbers[1] = 20;
numbers[2] = 30;
numbers[3] = 40;
numbers[4] = 50;
```

```
for(int i=0; i<numbers.length; i++){
    System.out.println(numbers[i]);
}
```

Output:

```
10
20
30
40
50
```

4. Example of Different Primitive Arrays

```
char[] letters = {'A', 'B', 'C'};
float[] marks = {75.5f, 82.3f, 91.0f};
boolean[] flags = {true, false, true};
```

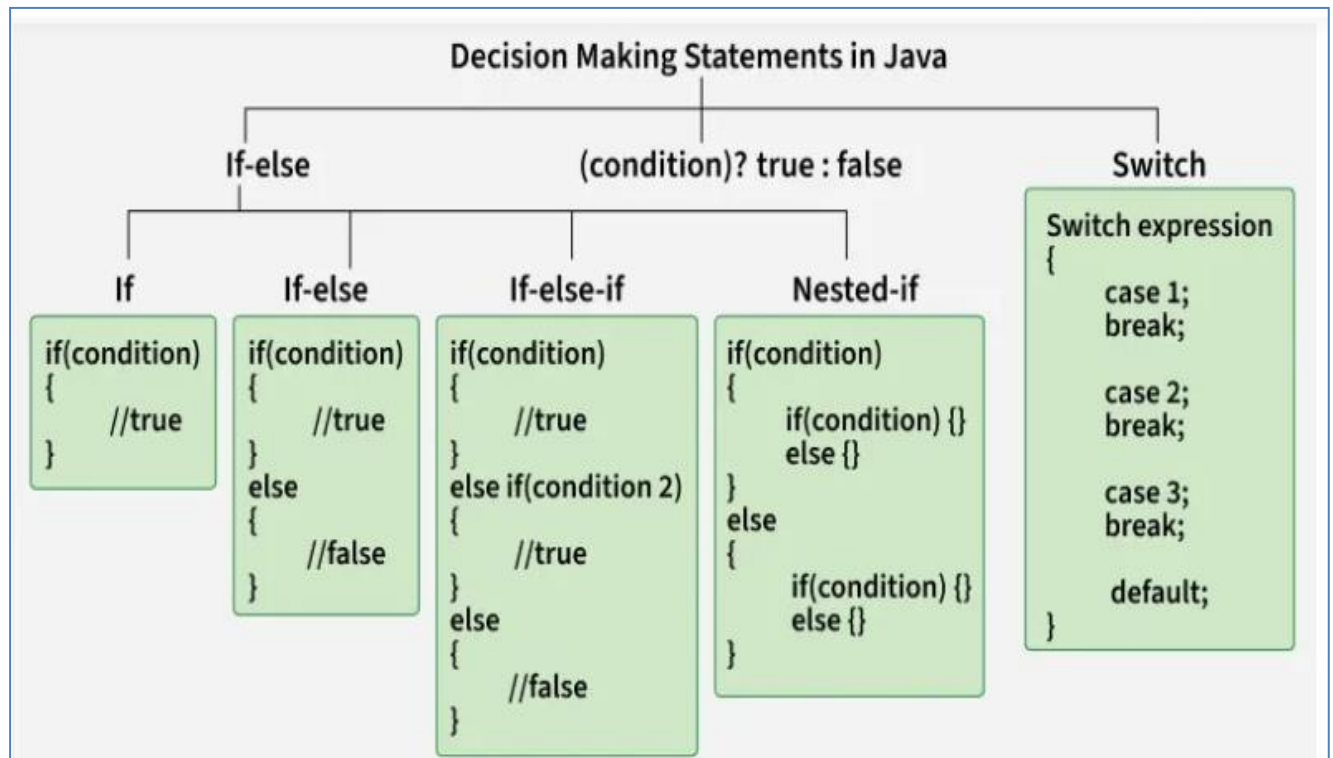
5. Note:

- All elements **must be of the same data type**.
- **Index starts from 0**.
- Arrays are **fixed in size**.
- Stored in **contiguous memory locations**.
- Default values depend on data type (e.g., 0, false, '\u0000').

Example:

```
boolean[] boolArr = new boolean[3];  
System.out.println(boolArr[0]); // Output: false
```

Decision & Control Statements



(Control Statements in Java)

Java compiler executes the code from top to bottom. The statements in the code are executed according to the order in which they appear. However, **Java** provides statements that can be used to control the flow of Java code. Such statements are called control flow statements. It is one of the fundamental features of Java, which provides a smooth flow of program.

Java provides three types of control flow statements.

1. Decision Making statements
 - if statements
 - switch statement
2. Loop statements
 - do while loop
 - while loop
 - for loop
 - for-each loop
3. Jump statements
 - break statement

- continue statement

Decision-Making statements:

As the name suggests, decision-making statements decide which statement to execute and when. Decision-making statements evaluate the Boolean expression and control the program flow depending upon the result of the condition provided. There are two types of decision-making statements in Java, i.e., If statement and switch statement.

1) If Statement:

In Java, the "if" statement is used to evaluate a condition. The control of the program is diverted depending upon the specific condition. The condition of the If statement gives a Boolean value, either true or false. In Java, there are four types of if-statements given below.

1. Simple if statement
2. if-else statement
3. if-else-if ladder
4. Nested if-statement

Let's understand the if-statements one by one.

1) Simple if statement:

It is the most basic statement among all control flow statements in Java. It evaluates a Boolean expression and enables the program to enter a block of code if the expression evaluates to true.

Syntax of if statement is given below.

1. **if**(condition) {
2. statement **1**; *//executes when condition is true*
3. }

Consider the following example in which we have used the **if** statement in the java code.

Example

1. **public class** Main {
2. **public static void** main(String[] args) {
3. **int** x = **10**;
4. **int** y = **12**;
5. **if**(x+y > **20**) {
6. System.out.println("x + y is greater than 20");
7. }
8. }
9. }

Output:

x + y is greater than 20

2) if-else statement

The **if-else statement** is an extension to the if-statement, which uses another block of code, i.e., else block. The else block is executed if the condition of the if-block is evaluated as false.

Syntax:

1. **if**(condition) {
2. statement **1**; *//executes when condition is true*
3. }
4. **else**{
5. statement **2**; *//executes when condition is false*
6. }

Consider the following example.

Example

1. **public class** Main {
2. **public static void** main(String[] args) {
3. **int** x = **10**;
4. **int** y = **12**;
5. **if**(x+y < **10**) {
6. System.out.println("x + y is less than 10");
7. } **else** {
8. System.out.println("x + y is greater than 20");
9. }
10. }
11. }

Output:

x + y is greater than 20

3) if-else-if ladder:

The if-else-if statement contains the if-statement followed by multiple else-if statements. In other words, we can say that it is the chain of if-else statements that create a decision tree where the program may enter in the block of code where the condition is true. We can also define an else statement at the end of the chain.

Syntax of if-else-if statement is given below.

1. **if**(condition **1**) {
2. statement **1**; *//executes when condition 1 is true*
3. }
4. **else if**(condition **2**) {
5. statement **2**; *//executes when condition 2 is true*
6. }
7. **else** {
8. statement **2**; *//executes when all the conditions are false*
9. }

Consider the following example.

Example

```
1. public class Main {
2.     public static void main(String[] args) {
3.         String city = "Delhi";
4.         if(city == "Meerut") {
5.             System.out.println("city is meerut");
6.         }else if (city == "Noida") {
7.             System.out.println("city is noida");
8.         }else if(city == "Agra") {
9.             System.out.println("city is agra");
10.        }else {
11.            System.out.println(city);
12.        }
13.    }
14. }
```

Output:

Delhi

4. Nested if-statement

In nested if-statements, the if statement can contain a **if** or **if-else** statement inside another if or else-if statement.

Syntax of Nested if-statement is given below.

```
1. if(condition 1) {
2.     statement 1; //executes when condition 1 is true
3.     if(condition 2) {
4.         statement 2; //executes when condition 2 is true
5.     }
6.     else{
7.         statement 2; //executes when condition 2 is false
8.     }
9. }
```

Consider the following example.

Example

```
1. public class Main {
2.     public static void main(String[] args) {
3.         String address = "Delhi, India";
4.
5.         if(address.endsWith("India")) {
6.             if(address.contains("Meerut")) {
7.                 System.out.println("Your city is Meerut");
8.             }else if(address.contains("Noida")) {
9.                 System.out.println("Your city is Noida");
10.            }else {
11.                System.out.println(address.split(",")[0]);
12.            }
13.        }else {
14.            System.out.println("You are not living in India");
15.        }
16.    }
17. }
```

```
15.     }
16. }
17. }
```

Output:

Delhi

Switch Statement:

In Java, **Switch statements** are similar to if-else-if statements. The switch statement contains multiple blocks of code called cases and a single case is executed based on the variable which is being switched. The switch statement is easier to use instead of if-else-if statements. It also enhances the readability of the program.

Points to be noted about switch statement:

- The case variables can be int, short, byte, char, or enumeration. String type is also supported since version 7 of Java
- Cases cannot be duplicate
- Default statement is executed when any of the case doesn't match the value of expression. It is optional.
- Break statement terminates the switch block when the condition is satisfied. It is optional, if not used, next case is executed.
- While using switch statements, we must notice that the case expression will be of the same type as the variable. However, it will also be a constant value.

The syntax to use the switch statement is given below.

```
1. switch (expression){
2.     case value1:
3.         statement1;
4.         break;
5.     .
6.     .
7.     .
8.     case valueN:
9.         statementN;
10.    break;
11.    default:
12.    default statement;
13. }
```

Consider the following example to understand the flow of the switch statement.

Example

```
1. public class Main {
2.     public static void main(String[] args) {
3.         int num = 2;
4.         switch (num){
5.             case 0:
6.                 System.out.println("number is 0");
7.                 break;
8.             case 1:
```

```

9.      System.out.println("number is 1");
10.     break;
11.     default:
12.      System.out.println(num);
13.  }
14. }
15. }

```

Output:

2

While using switch statements, we must notice that the case expression will be of the same type as the variable. However, it will also be a constant value. The switch permits only int, string, and Enum type variables to be used.

Loop Statements

In programming, sometimes we need to execute the block of code repeatedly while some condition evaluates to true. However, loop statements are used to execute the set of instructions in a repeated order. The execution of the set of instructions depends upon a particular condition.

In Java, we have three types of loops that execute similarly. However, there are differences in their syntax and condition checking time.

1. for loop
2. while loop
3. do-while loop

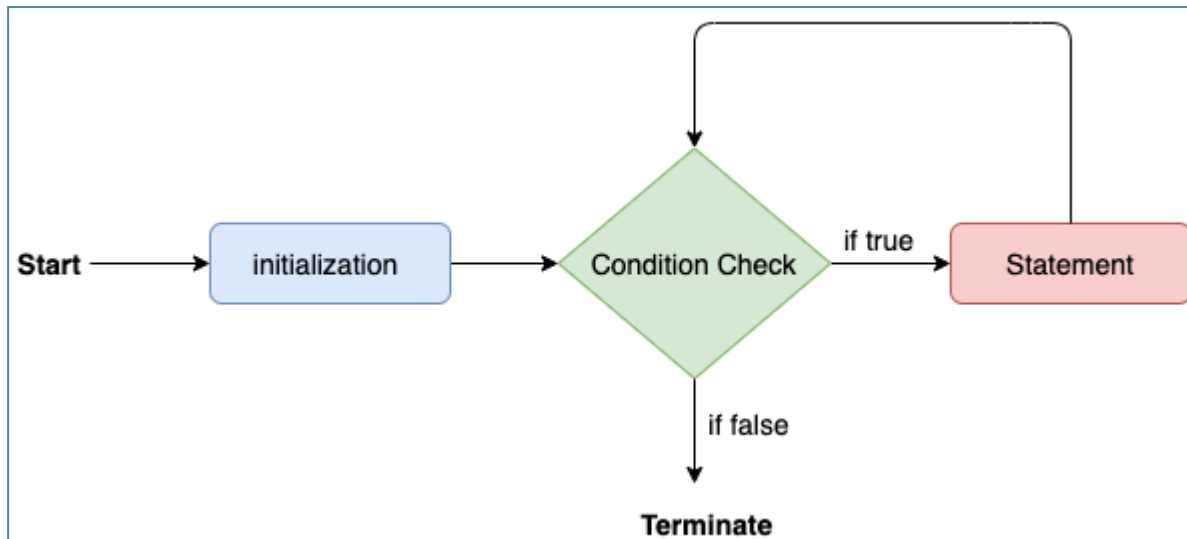
Let's understand the loop statements one by one.

Java for loop

In Java, **for loop** is similar to **C** and **C++**. It enables us to initialize the loop variable, check the condition, and increment/decrement in a single line of code. We use the for loop only when we exactly know the number of times, we want to execute the block of code.

1. **for**(initialization, condition, increment/decrement) {
2. *//block of statements*
3. }

The flow chart for the for-loop is given below.



Consider the following example to understand the proper functioning of the for loop in java.

Example

```

1. public class Main {
2.     public static void main(String[] args) {
3.         // TODO Auto-generated method stub
4.         int sum = 0;
5.         for(int j = 1; j<=10; j++) {
6.             sum = sum + j;
7.         }
8.         System.out.println("The sum of first 10 natural numbers is " + sum);
9.     }
10. }
  
```

Output:

The sum of first 10 natural numbers is 55

Java for-each loop

Java provides an enhanced for loop to traverse the data structures like array or collection. In the for-each loop, we don't need to update the loop variable. The syntax to use the for-each loop in java is given below.

```

1. for(data_type var : array_name/collection_name){
2.     //statements
3. }
  
```

Consider the following example to understand the functioning of the for-each loop in Java.

Example

```

1. public class Main {
2.     public static void main(String[] args) {
3.         String[] names = {"Java","C","C++","Python","JavaScript"};
4.         System.out.println("Printing the content of the array names:");
5.         for(String name:names) {
  
```

```
6.         System.out.println(name);
7.     }
8. }
9. }
```

Output:

Printing the content of the array names:

```
Java
C
C++
Python
JavaScript
```

Java while loop

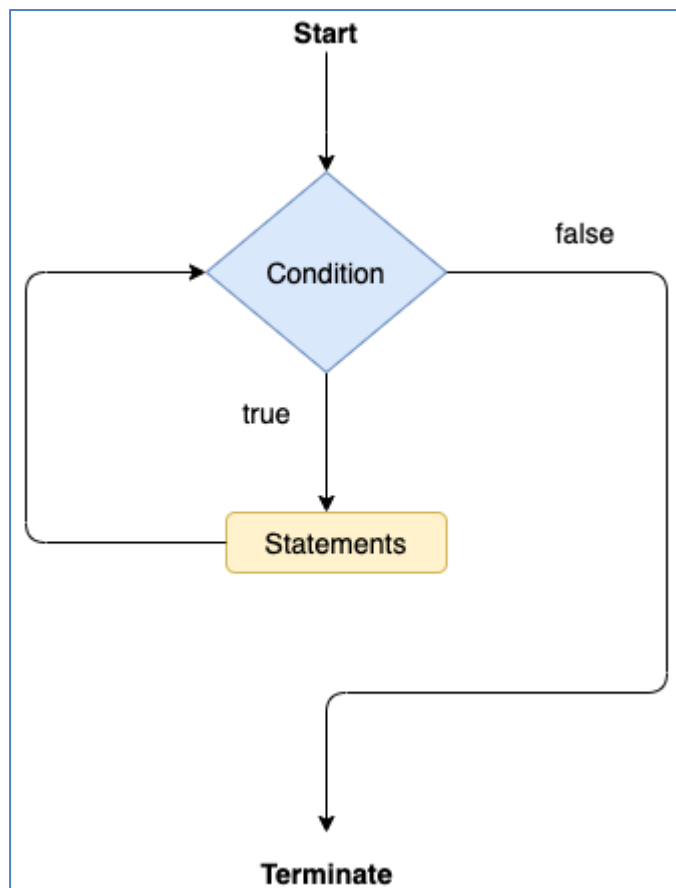
The **while loop** is also used to iterate over the number of statements multiple times. However, if we don't know the number of iterations in advance, it is recommended to use a while loop. Unlike for loop, the initialization and increment/decrement doesn't take place inside the loop statement in while loop.

It is also known as the entry-controlled loop since the condition is checked at the start of the loop. If the condition is true, then the loop body will be executed; otherwise, the statements after the loop will be executed.

The syntax of the while loop is given below.

```
1. while(condition){
2. //looping statements
3. }
```

The flow chart for the while loop is given in the following image.



Consider the following example.

Example

```
1. public class Main {  
2.     public static void main(String[] args) {  
3.         int i = 0;  
4.         System.out.println("Printing the list of first 10 even numbers");  
5.         while(i<=10) {  
6.             System.out.println(i);  
7.             i = i + 2;  
8.         }  
9.     }  
10. }
```

Output:

Printing the list of first 10 even numbers

0
2
4
6
8
10

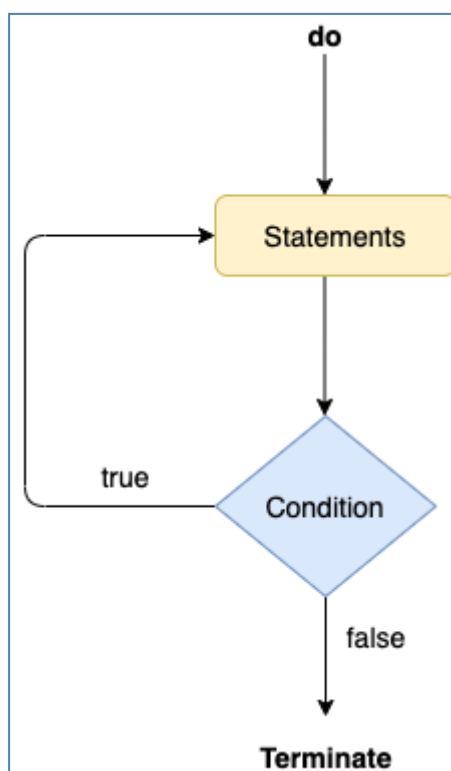
Java do-while loop

The **do-while** loop checks the condition at the end of the loop after executing the loop statements. When the number of iteration is not known and we have to execute the loop at least once, we can use do-while loop.

It is also known as the exit-controlled loop since the condition is not checked in advance. The syntax of the do-while loop is given below.

1. **do**
2. {
3. //statements
4. } **while** (condition);

The flow chart of the do-while loop is given in the following image.



Consider the following example to understand the functioning of the do-while loop in Java.

Example

1. **public class** Main {
2. **public static void** main(String[] args) {
3. **int** i = 0;
4. System.out.println("Printing the list of first 10 even numbers ");
5. **do** {
6. System.out.println(i);
7. i = i + 2;
8. } **while** (i <= 10);
9. }
10. }

Output:

Printing the list of first 10 even numbers

0
2
4
6
8
10

Jump Statements

Jump statements are used to transfer the control of the program to the specific statements. In other words, jump statements transfer the execution control to the other part of the program. There are two types of jump statements in Java, i.e., break and continue.

Java break statement

As the name suggests, the **break statement** is used to break the current flow of the program and transfer the control to the next statement outside a loop or switch statement. However, it breaks only the inner loop in the case of the nested loop.

The break statement cannot be used independently in the Java program, i.e., it can only be written inside the loop or switch statement.

The break statement example with for loop

Consider the following example in which we have used the break statement with the for loop.

Example

```
1. public class Main {  
2.     public static void main(String[] args) {  
3.         for(int i = 0; i<= 10; i++) {  
4.             System.out.println(i);  
5.             if(i==6) {  
6.                 break;  
7.             }  
8.         }  
9.     }  
10. }
```

Output:

0
1
2
3
4
5
6

break statement example with labeled for loop

Example

```
1. public class Main {
2.     public static void main(String[] args) {
3.         a:
4.         for(int i = 0; i<= 10; i++) {
5.             b:
6.             for(int j = 0; j<=15;j++) {
7.                 c:
8.                 for (int k = 0; k<=20; k++) {
9.                     System.out.println(k);
10.                    if(k==5) {
11.                        break a;
12.                    }
13.                }
14.            }
15.        }
16.    }
17. }
18. }
```

Output:

```
0
1
2
3
4
5
```

Java continue statement

Unlike break statement, the **continue statement** doesn't break the loop, whereas, it skips the specific part of the loop and jumps to the next iteration of the loop immediately.

Consider the following example to understand the functioning of the continue statement in Java.

Example

```
1. public class Main {
2.     public static void main(String[] args) {
3.         for(int i = 0; i<= 2; i++) {
4.
5.             for (int j = i; j<=5; j++) {
6.
7.                 if(j == 4) {
8.                     continue;
9.                 }
10.                System.out.println(j);
11.            }
12.        }
13.    }
14. }
```

Output:

0
1
2
3
5
1
2
3
5
2
3
5

Different Operators

Operators are an essential part of any programming language. In Java, operator is a symbol that is used to perform operations. For example: +, -, *, / etc. These are essential for performing different types of operations on **variables** and values. In this section, we will discuss **different types of operators used in Java programming**.

There are mainly eight types of operators in Java:

Unary Operator

Arithmetic Operator

Relational Operator

Ternary Operator

Assignment Operator

Bitwise Operator

Logical Operator

Shift Operator

1. Unary Operators

Require **one operand**.

Used for **increment, decrement, negation, and logical NOT**.

Operator	Name	Description
+	Unary Plus	Indicates positive value
-	Unary Minus	Converts positive to negative
!	Logical NOT	Reverses boolean value
++	Increment	Increases value by 1 (x++, ++x)
--	Decrement	Decreases value by 1 (x--, --x)

Example:

```
int x = 10;  
System.out.println(x++); // 10  
System.out.println(++x); // 12
```

2. Arithmetic Operators

Perform **basic math operations**.

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus (remainder)

Example:

```
int a=10,b=5;  
System.out.println(a+b); //15  
System.out.println(a%b); //0
```

3. Shift Operators

Used to **shift bits** of numbers.

Operator	Name	Description
----------	------	-------------

<<	Left Shift	Shifts bits left
>>	Right Shift	Shifts bits right
>>>	Unsigned Right Shift	Fills left bits with 0

Example:

10 << 2 → 40, 10 >> 2 → 2

4. Relational Operators

Compare two values; return **true/false**.

Operator	Description
<, >	Less / Greater
<=, >=	Less/Greater or Equal
==, !=	Equal / Not Equal

Example:

a < b, a == b, a != b

5. Bitwise Operators

Operate on bits (integers only).

Operator	Description
&	Bitwise AND
^	Bitwise XOR
~	Bitwise Complement

Example:

a & b, a | b, ~a

6. Logical Operators

Used with boolean values.

Operator	Description
----------	-------------

&& Logical AND

,

! Logical NOT

Example:

```
if(a > b && a < c)
    System.out.println("True");
```

7. Ternary Operator

Used as **short form of if-else**.

Syntax: condition ? expr1 : expr2

Example:

```
int min = (a < b) ? a : b;
```

8. Assignment Operators

Used to **assign values**.

Operator	Description
=	Assign value
+=	Add and assign
-=	Subtract and assign
*=	Multiply and assign
/=	Divide and assign
%=	Modulus and assign

Example:

```
int a = 10;
a += 5; // a = 15
```

9. Operator Precedence (High → Low)

1. Unary (++ , -- , ! , ~)
2. Multiplicative (* , / , %)
3. Additive (+ , -)

4. Shift (<<, >>, >>>)
5. Relational (<, >, <=, >=, instanceof)
6. Equality (==, !=)
7. Bitwise (&, ^, |)
8. Logical (&&, ||)
9. Ternary (?:)
10. Assignment (=, +=, -=, *=)

Overall :

- **Unary** → One operand
- **Arithmetic** → Math operations
- **Shift** → Bit-level shift
- **Relational** → Comparisons
- **Bitwise** → Bit manipulation
- **Logical** → Boolean conditions
- **Ternary** → One-line if-else
- **Assignment** → Value assignment